

Investigacion Operativa

Recorrido de grafos y camino mínimo

Facundo Gutiérrez - Nazareno Faillace

Departamento de Matemática, FCEN, UBA

Recorrido de Grafos - BFS

Dado un grafo $G = (\mathbb{V}, \mathbb{E})$, queremos recorrer sus vértices. En una primera instancia vamos a asumir que es *conexo*. Según cómo decidamos recorrer el grafo vamos a poder **distinguir distintas propiedades estructurales** del mismo.

Recorrido de Grafos - BFS

Dado un grafo $G = (\mathbb{V}, \mathbb{E})$, queremos recorrer sus vértices. En una primera instancia vamos a asumir que es *conexo*. Según cómo decidamos recorrer el grafo vamos a poder **distinguir distintas propiedades estructurales** del mismo.

En un recorrido **BFS** (*Breadth-First Search*) vamos a distinguir a un nodo en particular donde comenzaremos el recorrido del grafo. Llamémosle s a dicho nodo.

Recorrido de Grafos - BFS

Dado un grafo $G = (\mathbb{V}, \mathbb{E})$, queremos recorrer sus vértices. En una primera instancia vamos a asumir que es *conexo*. Según cómo decidamos recorrer el grafo vamos a poder **distinguir distintas propiedades estructurales** del mismo.

En un recorrido **BFS** (*Breadth-First Search*) vamos a distinguir a un nodo en particular donde comenzaremos el recorrido del grafo. Llamémosle s a dicho nodo.

Comenzamos visitando el nodo s , luego visitamos a todos los vecinos de s (que son los nodos que están a distancia 1 de s), después visitamos a los vecinos de estos vecinos (que son los nodos que están a distancia 2 de s), y así

A lo largo del algoritmo utilizaremos:

A lo largo del algoritmo utilizaremos:

- S : indica la capa actual de vértices que estamos visitando.
- W : indica la capa siguiente de vértices por visitar.
- **visto**: un arreglo auxiliar que indica qué vértices ya fueron procesados.
- s : vértice inicial en el que comenzamos el recorrido.

Algoritmo BFS

BFS(G, s):

1. $\forall v \in V : \text{visto}[v] \leftarrow \text{Falso}$
2. $\text{visto}[s] \leftarrow \text{Verdadero}, S \leftarrow \{s\}, W \leftarrow \emptyset$
3. Mientras $S \neq \emptyset$:
 - 3.1 $\forall v \in S$:
 - 3.1.1 $\forall w \in \{u \in \mathcal{N}(v) : \text{visto}[u] = \text{Falso}\}$:
Agregar w a W
 $\text{visto}[w] \leftarrow \text{Verdadero}$
 - 3.2 $S \leftarrow W$
 - 3.3 $W \leftarrow \emptyset$

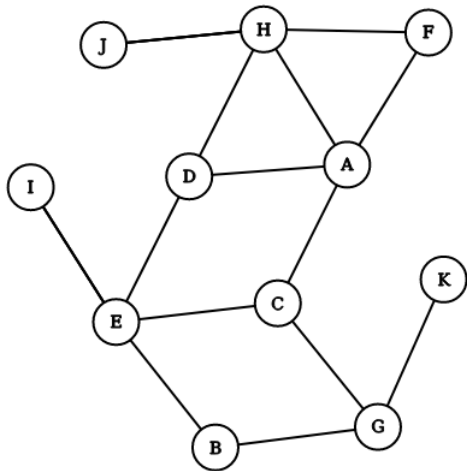
Algoritmo BFS

BFS(G, s):

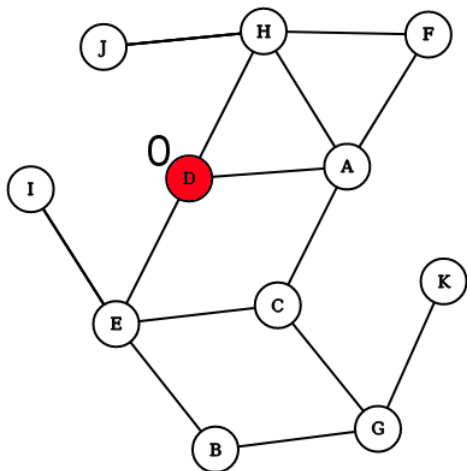
1. $\forall v \in V : \text{visto}[v] \leftarrow \text{Falso}$
2. $\text{visto}[s] \leftarrow \text{Verdadero}, S \leftarrow \{s\}, W \leftarrow \emptyset$
3. Mientras $S \neq \emptyset$:
 - 3.1 $\forall v \in S$:
 - 3.1.1 $\forall w \in \{u \in \mathcal{N}(v) : \text{visto}[u] = \text{Falso}\}$:
Agregar w a W
 $\text{visto}[w] \leftarrow \text{Verdadero}$
 - 3.2 $S \leftarrow W$
 - 3.3 $W \leftarrow \emptyset$

¿Cómo podemos modificarlo para **calcular la distancia** a s ?

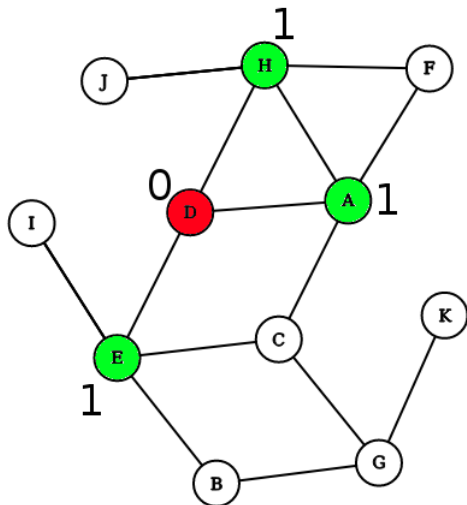
Recorrido de Grafos - BFS



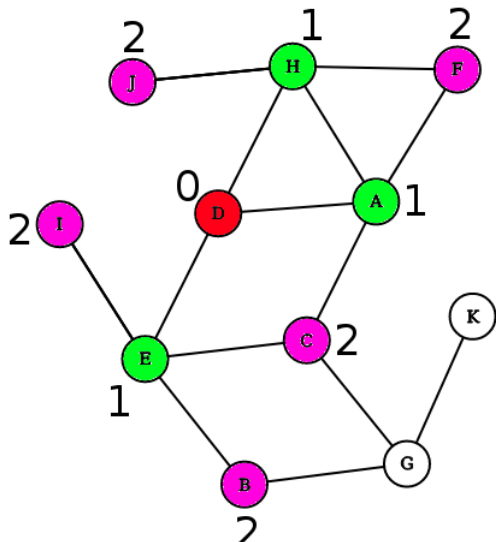
Recorrido de Grafos - BFS



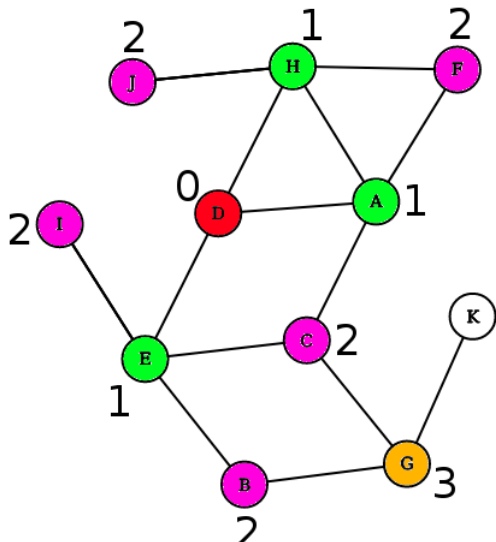
Recorrido de Grafos - BFS



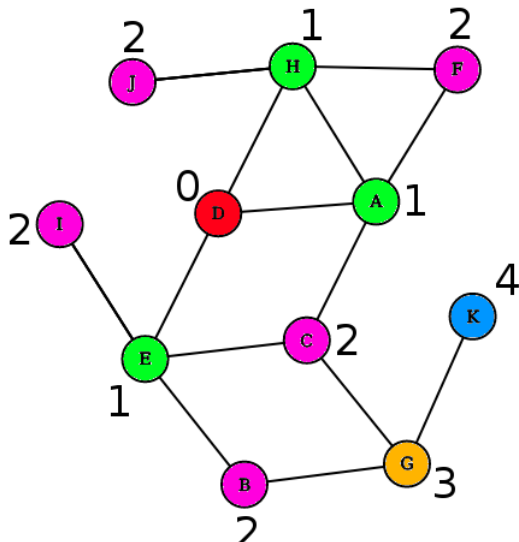
Recorrido de Grafos - BFS



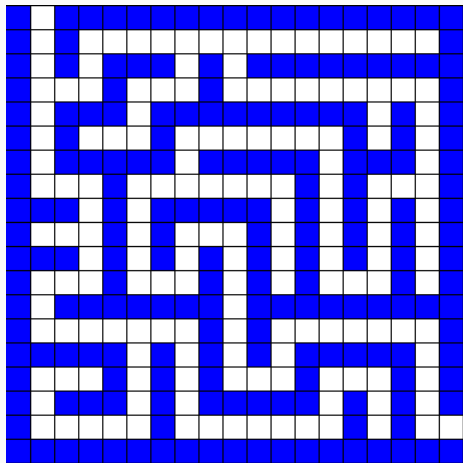
Recorrido de Grafos - BFS



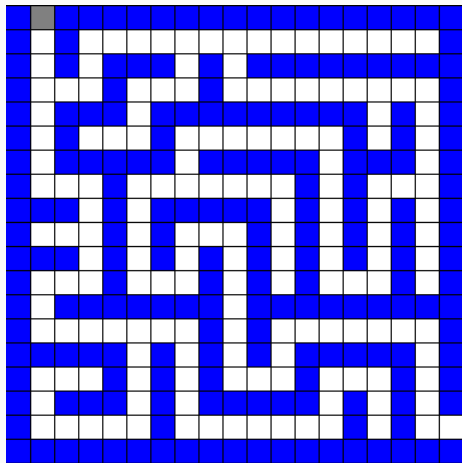
Recorrido de Grafos - BFS



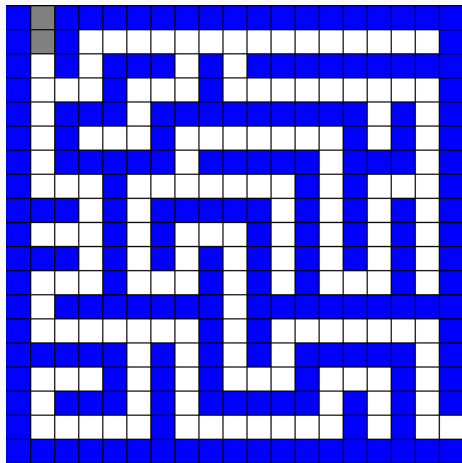
Recorrido de Grafos - BFS



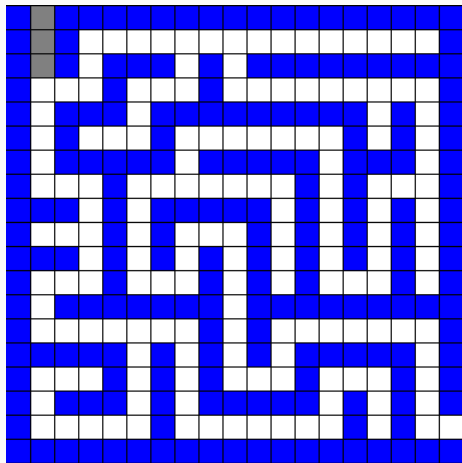
Recorrido de Grafos - BFS



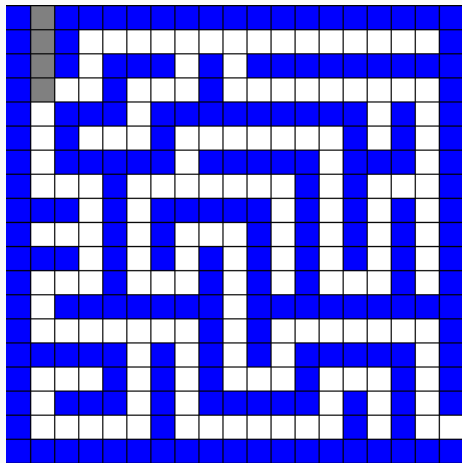
Recorrido de Grafos - BFS



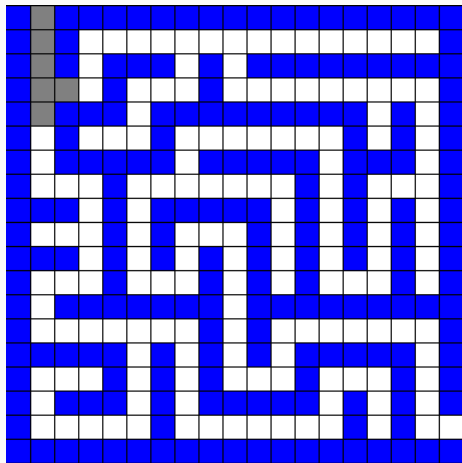
Recorrido de Grafos - BFS



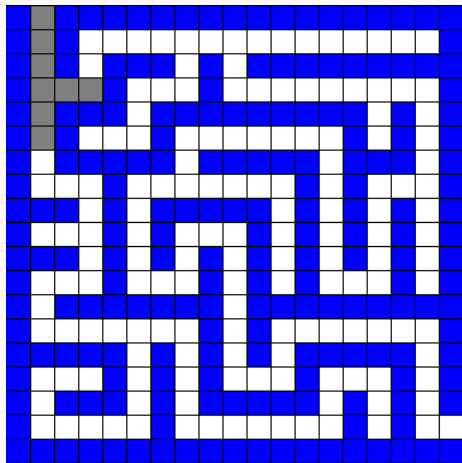
Recorrido de Grafos - BFS



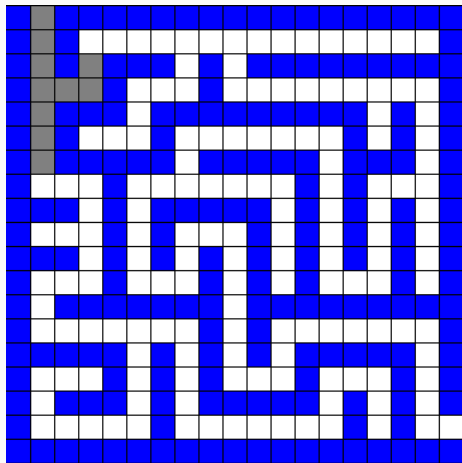
Recorrido de Grafos - BFS



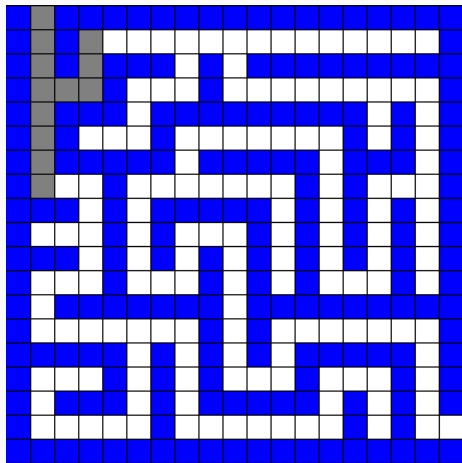
Recorrido de Grafos - BFS



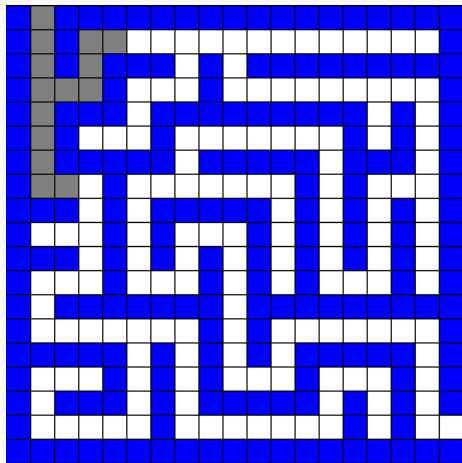
Recorrido de Grafos - BFS



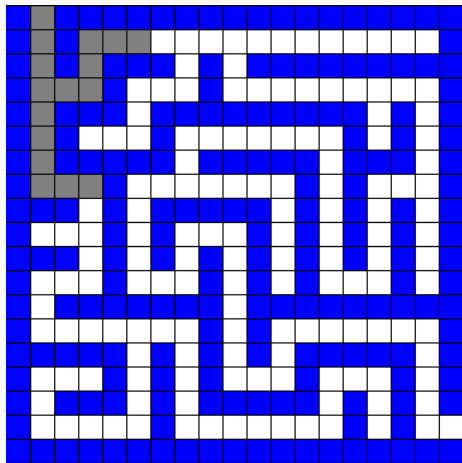
Recorrido de Grafos - BFS



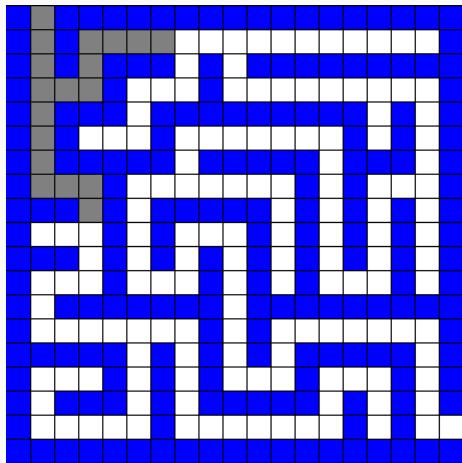
Recorrido de Grafos - BFS



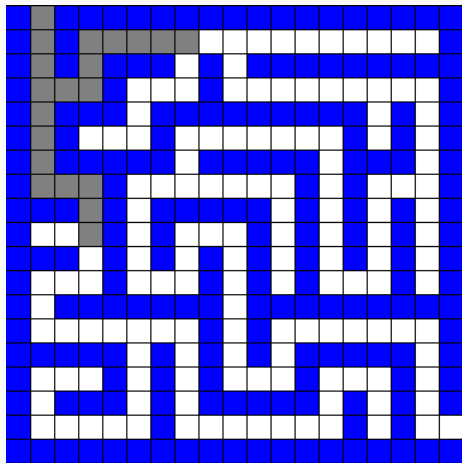
Recorrido de Grafos - BFS



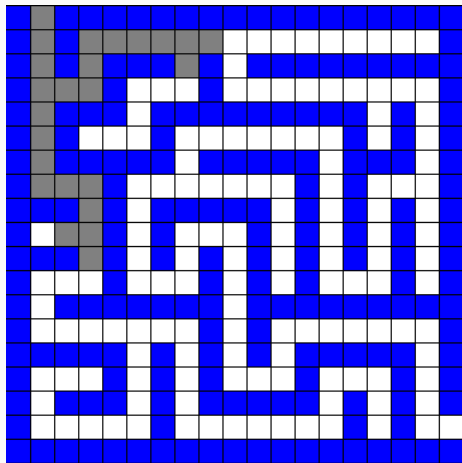
Recorrido de Grafos - BFS



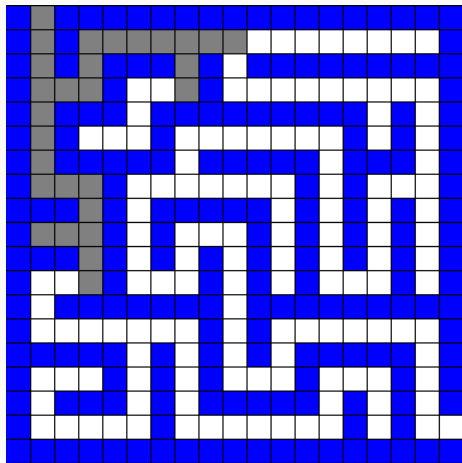
Recorrido de Grafos - BFS



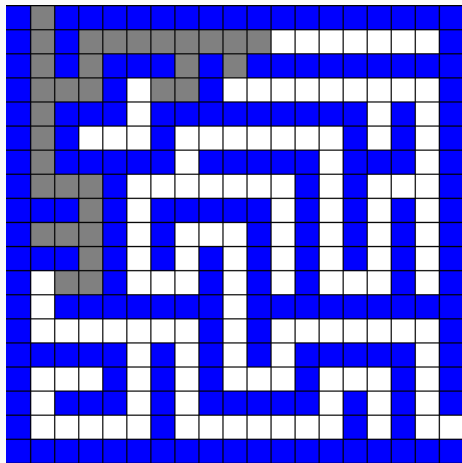
Recorrido de Grafos - BFS



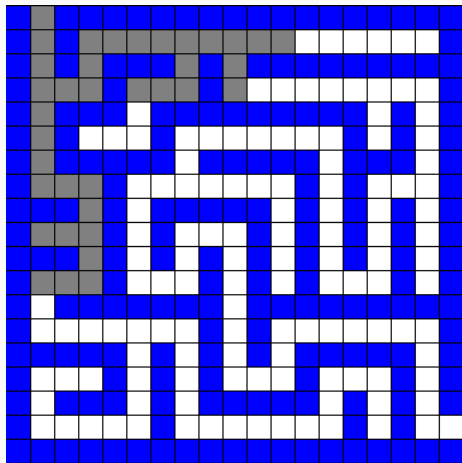
Recorrido de Grafos - BFS



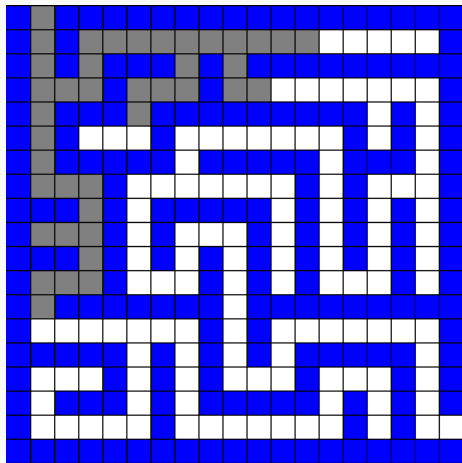
Recorrido de Grafos - BFS



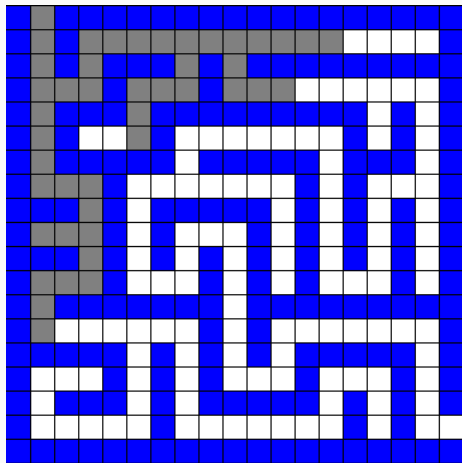
Recorrido de Grafos - BFS



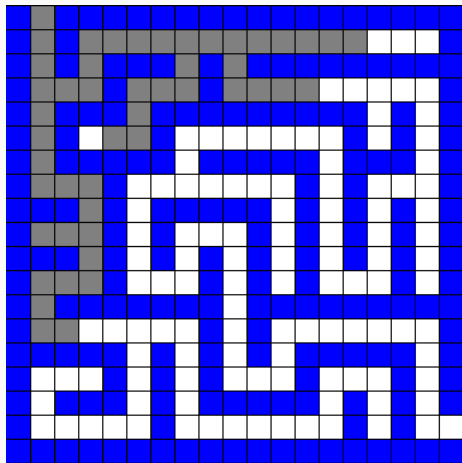
Recorrido de Grafos - BFS



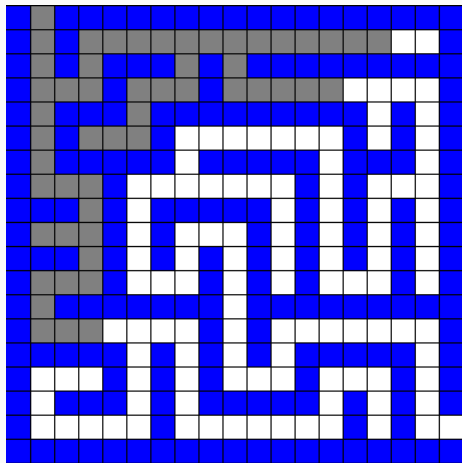
Recorrido de Grafos - BFS



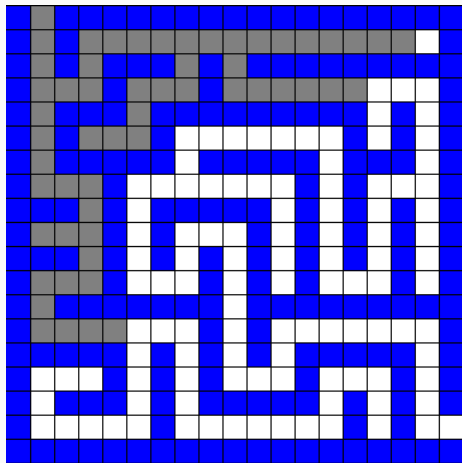
Recorrido de Grafos - BFS



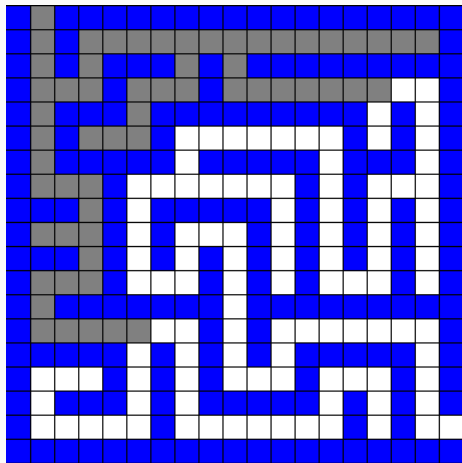
Recorrido de Grafos - BFS



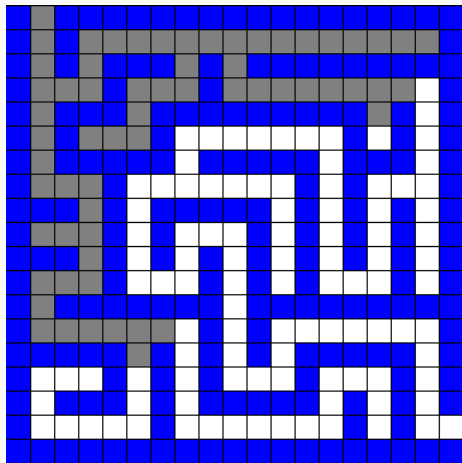
Recorrido de Grafos - BFS



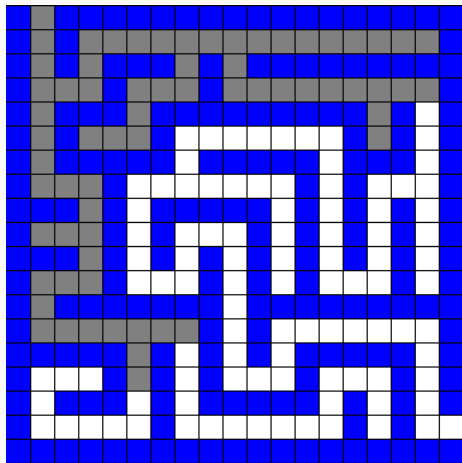
Recorrido de Grafos - BFS



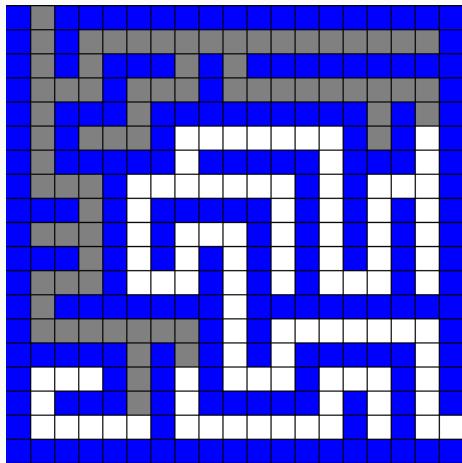
Recorrido de Grafos - BFS



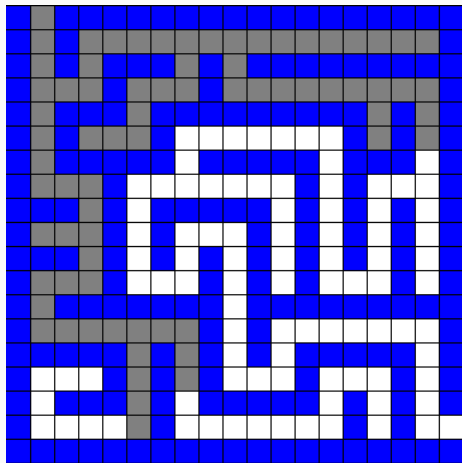
Recorrido de Grafos - BFS



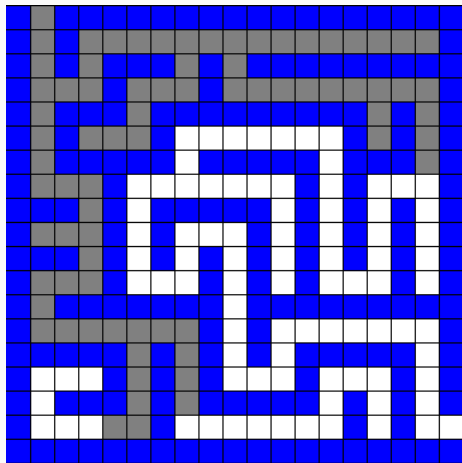
Recorrido de Grafos - BFS



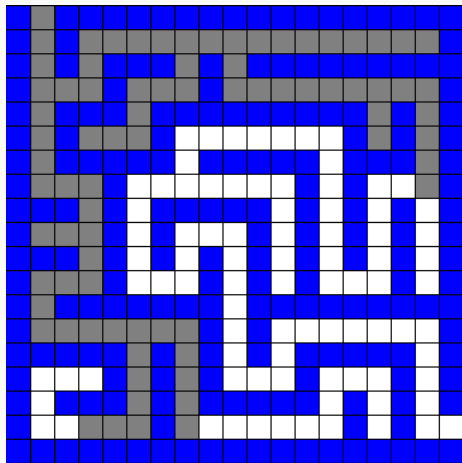
Recorrido de Grafos - BFS



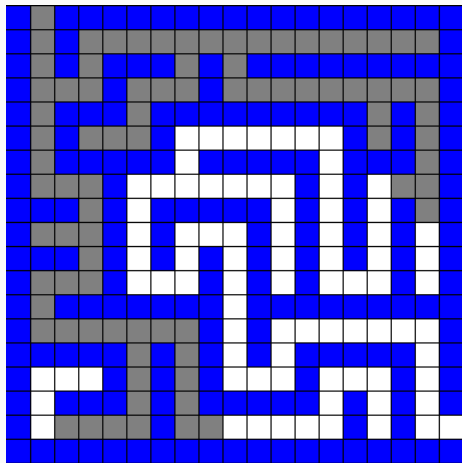
Recorrido de Grafos - BFS



Recorrido de Grafos - BFS



Recorrido de Grafos - BFS



Recorrido de Grafos - BFS

Cada nodo se agrega a lo sumo una vez a S . Cuando procesamos a un nodo en S analizamos la situación de todos sus vecinos. Por lo tanto, en total **hacemos** $\mathcal{O}(|V| + |E|)$ operaciones (lo cual es lineal en el grafo).

Recorrido de Grafos - BFS

Cada nodo se agrega a lo sumo una vez a S . Cuando procesamos a un nodo en S analizamos la situación de todos sus vecinos. Por lo tanto, en total **hacemos** $\mathcal{O}(|V| + |E|)$ operaciones (lo cual es lineal en el grafo).

Además de simplemente recorrer un grafo en general, los usos más comunes de BFS (o alguna modificación del mismo) involucran:

Recorrido de Grafos - BFS

Cada nodo se agrega a lo sumo una vez a S . Cuando procesamos a un nodo en S analizamos la situación de todos sus vecinos. Por lo tanto, en total **hacemos** $\mathcal{O}(|V| + |E|)$ operaciones (lo cual es lineal en el grafo).

Además de simplemente recorrer un grafo en general, los usos más comunes de BFS (o alguna modificación del mismo) involucran:

- Calcular distancias desde una fuente

Recorrido de Grafos - BFS

Cada nodo se agrega a lo sumo una vez a S . Cuando procesamos a un nodo en S analizamos la situación de todos sus vecinos. Por lo tanto, en total **hacemos** $\mathcal{O}(|V| + |E|)$ operaciones (lo cual es lineal en el grafo).

Además de simplemente recorrer un grafo en general, los usos más comunes de BFS (o alguna modificación del mismo) involucran:

- Calcular distancias desde una fuente
- Chequear si un cierto grafo es bipartito

Recorrido de Grafos - BFS

Cada nodo se agrega a lo sumo una vez a S . Cuando procesamos a un nodo en S analizamos la situación de todos sus vecinos. Por lo tanto, en total **hacemos** $\mathcal{O}(|V| + |E|)$ operaciones (lo cual es lineal en el grafo).

Además de simplemente recorrer un grafo en general, los usos más comunes de BFS (o alguna modificación del mismo) involucran:

- Calcular distancias desde una fuente
- Chequear si un cierto grafo es bipartito
- Calcular componentes conexas

Recorrido de Grafos - BFS

Cada nodo se agrega a lo sumo una vez a S . Cuando procesamos a un nodo en S analizamos la situación de todos sus vecinos. Por lo tanto, en total **hacemos** $\mathcal{O}(|V| + |E|)$ operaciones (lo cual es lineal en el grafo).

Además de simplemente recorrer un grafo en general, los usos más comunes de BFS (o alguna modificación del mismo) involucran:

- Calcular distancias desde una fuente
- Chequear si un cierto grafo es bipartito
- Calcular componentes conexas

¿Qué podemos hacer si el grafo no es *conexo*?

Veamos otra forma de recorrer a $G = (\mathbb{V}, \mathbb{E})$ de una forma que ya fuimos utilizando a lo largo de la materia (por ejemplo en Branch & Bound).

Veamos otra forma de recorrer a $G = (\mathbb{V}, \mathbb{E})$ de una forma que ya fuimos utilizando a lo largo de la materia (por ejemplo en Branch & Bound).

Vamos a distinguir a un nodo en particular que indica el nodo actual del recorrido. Llamémosle v a dicho nodo.

Recorrido de Grafos - DFS

Veamos otra forma de recorrer a $G = (\mathbb{V}, \mathbb{E})$ de una forma que ya fuimos utilizando a lo largo de la materia (por ejemplo en Branch & Bound).

Vamos a distinguir a un nodo en particular que indica el nodo actual del recorrido. Llamémosle v a dicho nodo.

En un recorrido **DFS** (*Depth-First Search*) comenzamos visitando el nodo v , luego visitamos a algún vecino y seguimos un recorrido DFS desde allí. Así hasta que todos los vecinos de v sean visitados. Una vez que todos los vecinos de v están visitados, se da por finalizado el recorrido desde v .

Recorrido de Grafos - DFS

Veamos otra forma de recorrer a $G = (\mathbb{V}, \mathbb{E})$ de una forma que ya fuimos utilizando a lo largo de la materia (por ejemplo en Branch & Bound).

Vamos a distinguir a un nodo en particular que indica el nodo actual del recorrido. Llamémosle v a dicho nodo.

En un recorrido **DFS** (*Depth-First Search*) comenzamos visitando el nodo v , luego visitamos a algún vecino y seguimos un recorrido DFS desde allí. Así hasta que todos los vecinos de v sean visitados. Una vez que todos los vecinos de v están visitados, se da por finalizado el recorrido desde v .

De alguna forma, podemos pensar que se va expandiendo ordenadamente en una dirección todo lo que puede.

Por la naturaleza del algoritmo, las implementaciones usuales del recorrido DFS de un grafo son recursivas. A lo largo del algoritmo utilizaremos el arreglo auxiliar **visto** de manera similar.

Algoritmo DFS

DFS(G):

1. $\forall v \in \mathbb{V} : \mathbf{visto}[v] \leftarrow \textit{Falso}$
2. $\forall v \in \mathbb{V} : \text{Si } \mathbf{visto}[v] = \textit{Falso} :$
 - 2.1 DFS-VISITA($G, v, \mathbf{visto}$)

Algoritmo DFS

DFS(G):

1. $\forall v \in \mathbb{V} : \mathbf{visto}[v] \leftarrow \textit{Falso}$
2. $\forall v \in \mathbb{V} : \text{Si } \mathbf{visto}[v] = \textit{Falso} :$
 - 2.1 DFS-VISITA($G, v, \mathbf{visto}$)

DFS-VISITA($G, v, \mathbf{visto}$):

1. $\mathbf{visto}[v] \leftarrow \textit{Verdadero}$
2. $\forall w \in \{u \in N(v) : \mathbf{visto}[u] = \textit{Falso}\}$
 - 2.1 DFS-VISITA($G, w, \mathbf{visto}$)

Algoritmo DFS

DFS(G):

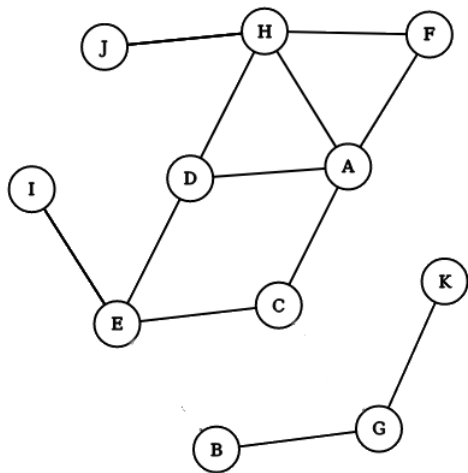
1. $\forall v \in V : \mathbf{visto}[v] \leftarrow Falso$
2. $\forall v \in V : \text{Si } \mathbf{visto}[v] = Falso :$
 - 2.1 DFS-VISITA($G, v, \mathbf{visto}$)

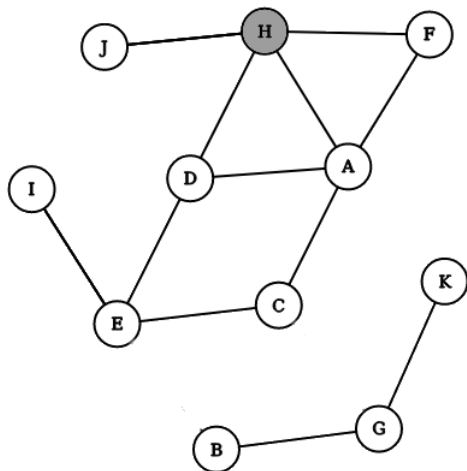
DFS-VISITA($G, v, \mathbf{visto}$):

1. $\mathbf{visto}[v] \leftarrow Verdadero$
2. $\forall w \in \{u \in N(v) : \mathbf{visto}[u] = Falso\}$
 - 2.1 DFS-VISITA($G, w, \mathbf{visto}$)

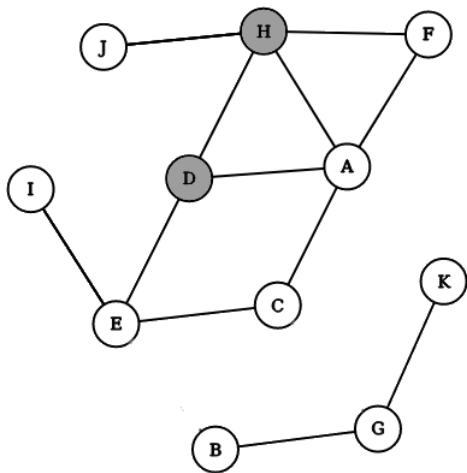
¿Cómo podemos modificarlo para encontrar la cantidad de componentes conexas del grafo?

Recorrido de Grafos - DFS

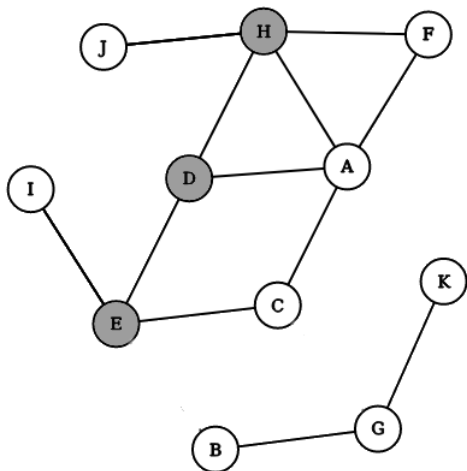




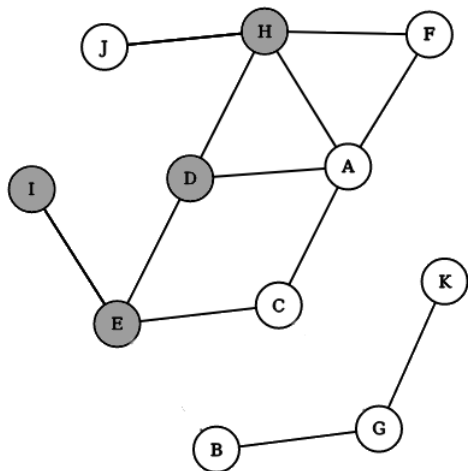
Recorrido de Grafos - DFS



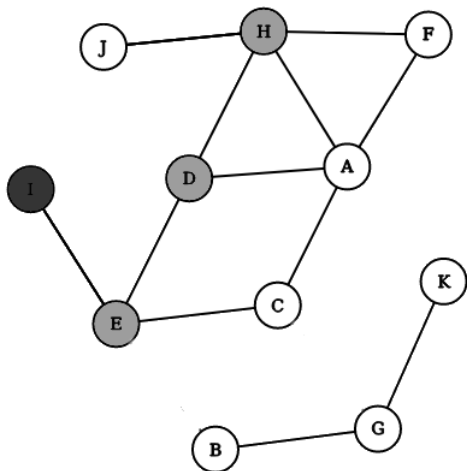
Recorrido de Grafos - DFS



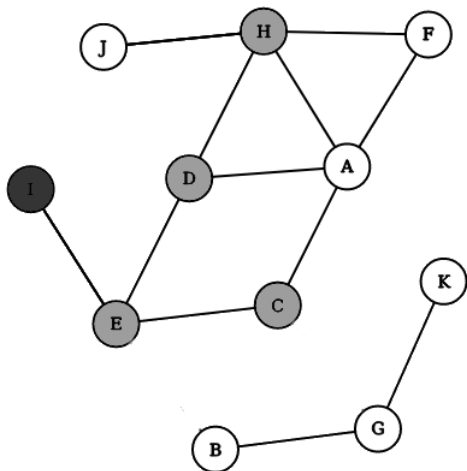
Recorrido de Grafos - DFS



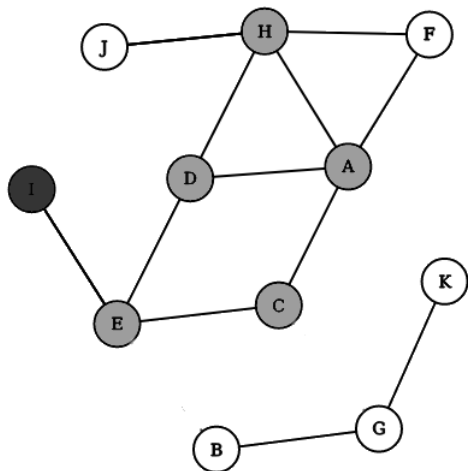
Recorrido de Grafos - DFS



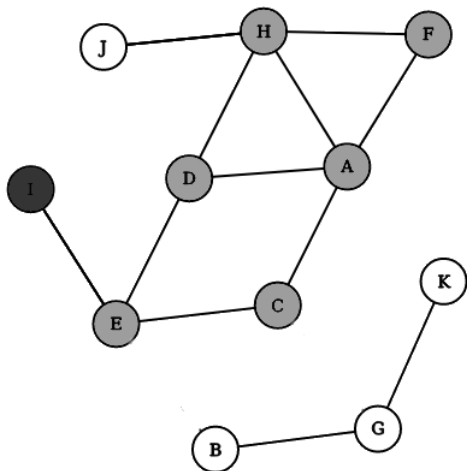
Recorrido de Grafos - DFS



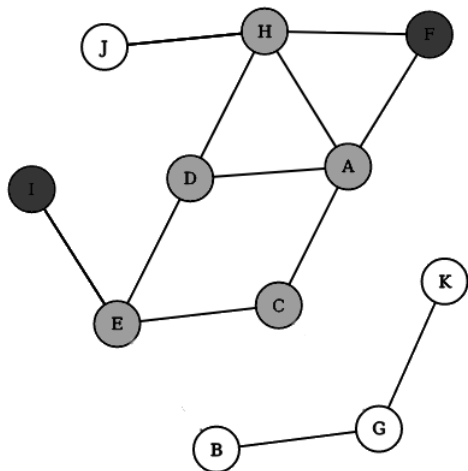
Recorrido de Grafos - DFS



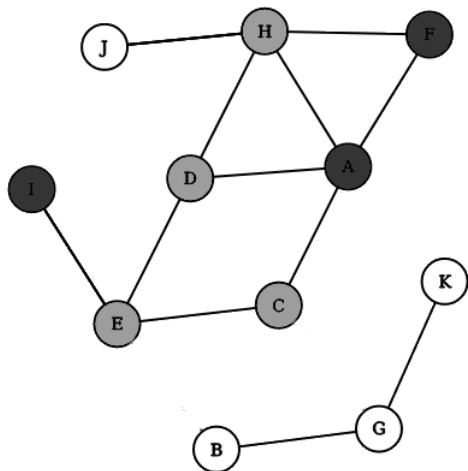
Recorrido de Grafos - DFS



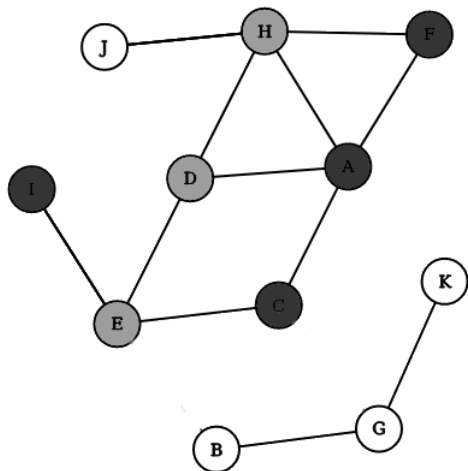
Recorrido de Grafos - DFS



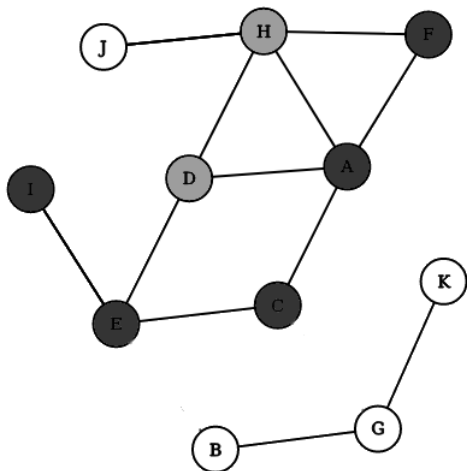
Recorrido de Grafos - DFS



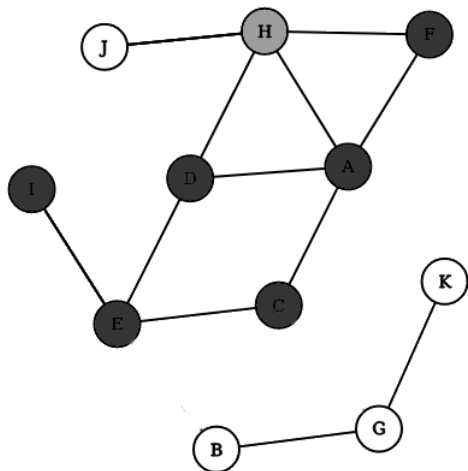
Recorrido de Grafos - DFS



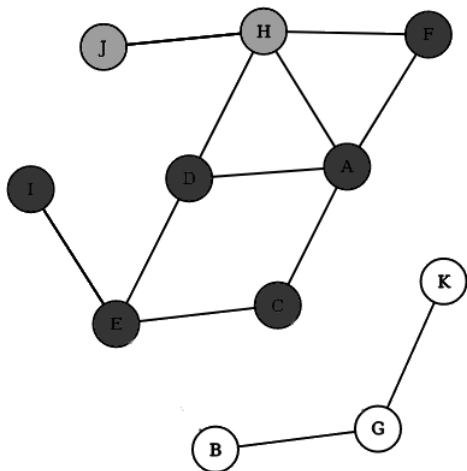
Recorrido de Grafos - DFS



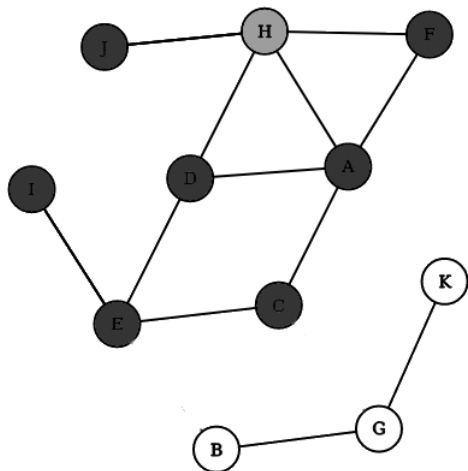
Recorrido de Grafos - DFS



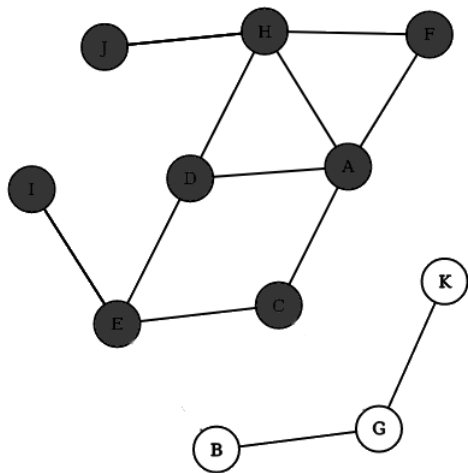
Recorrido de Grafos - DFS



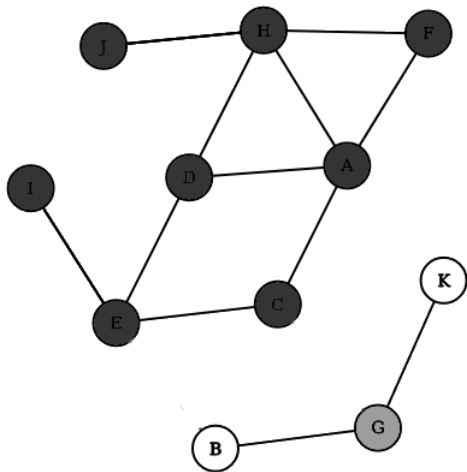
Recorrido de Grafos - DFS



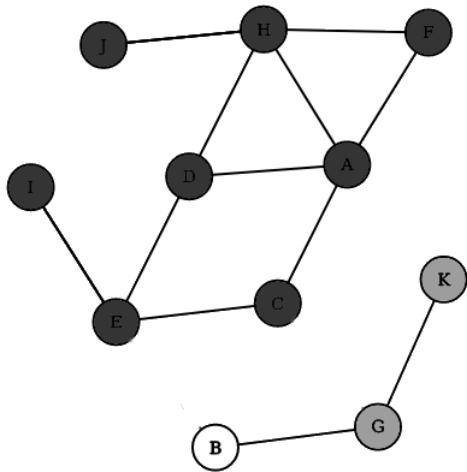
Recorrido de Grafos - DFS



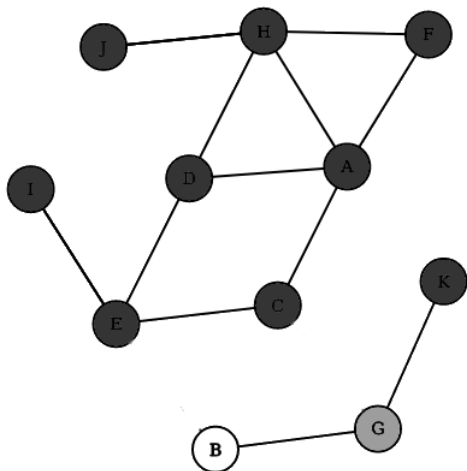
Recorrido de Grafos - DFS



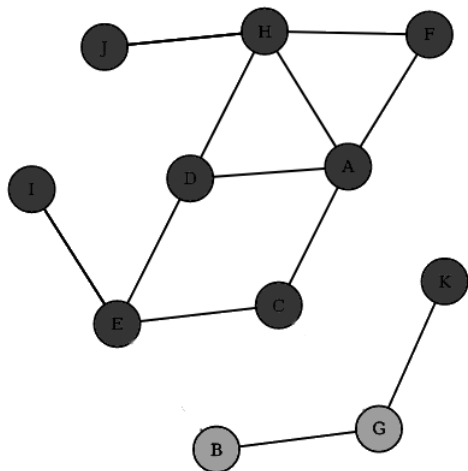
Recorrido de Grafos - DFS



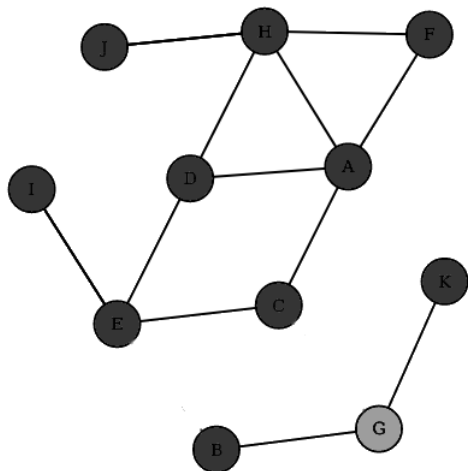
Recorrido de Grafos - DFS



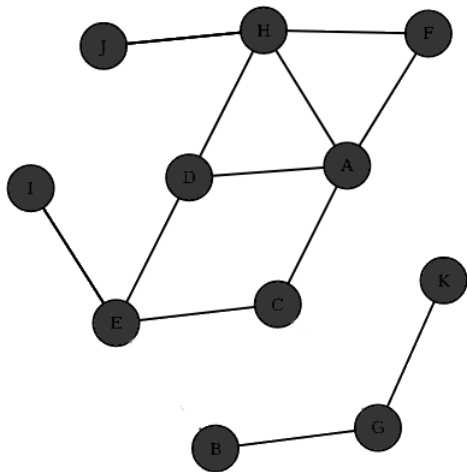
Recorrido de Grafos - DFS



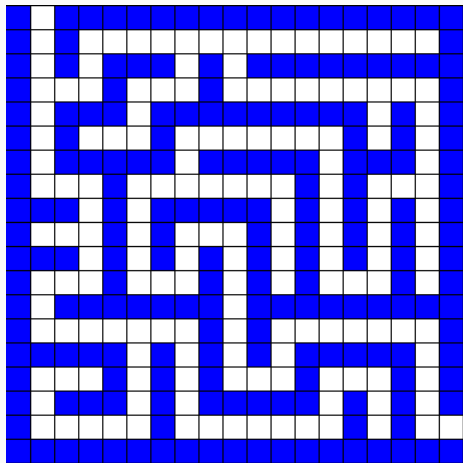
Recorrido de Grafos - DFS



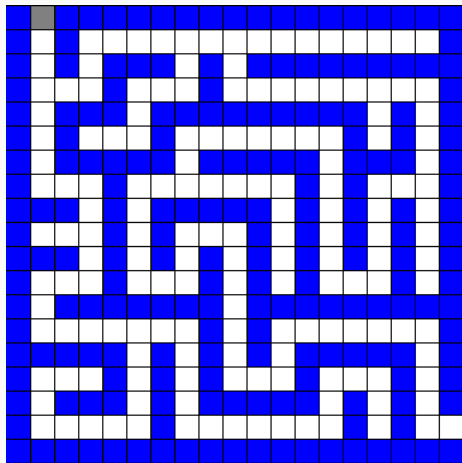
Recorrido de Grafos - DFS



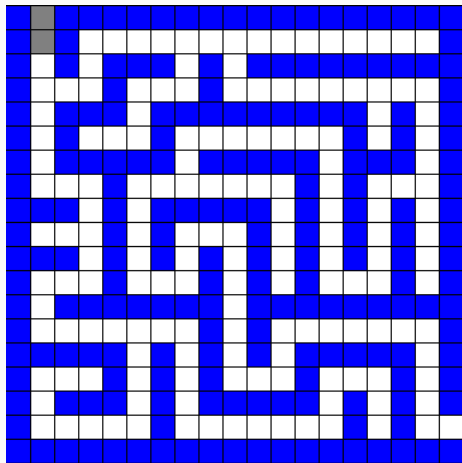
Recorrido de Grafos - DFS



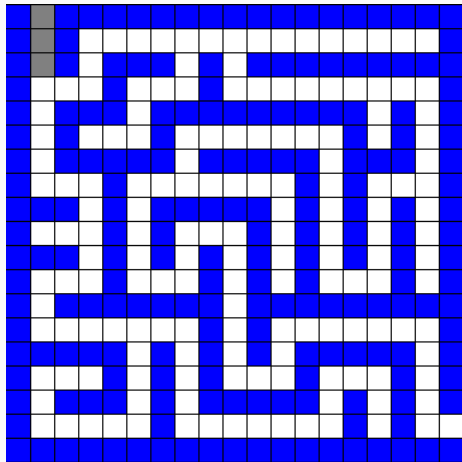
Recorrido de Grafos - DFS



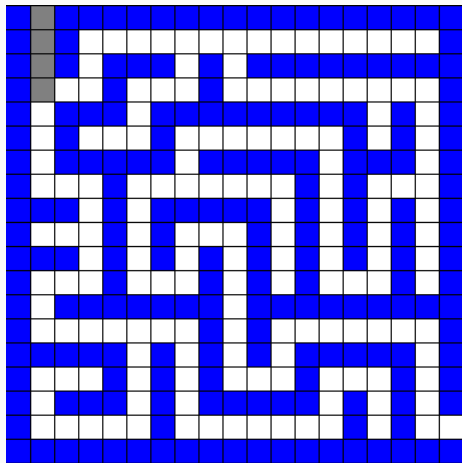
Recorrido de Grafos - DFS



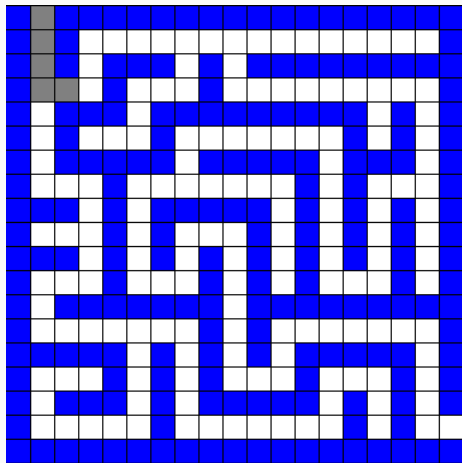
Recorrido de Grafos - DFS



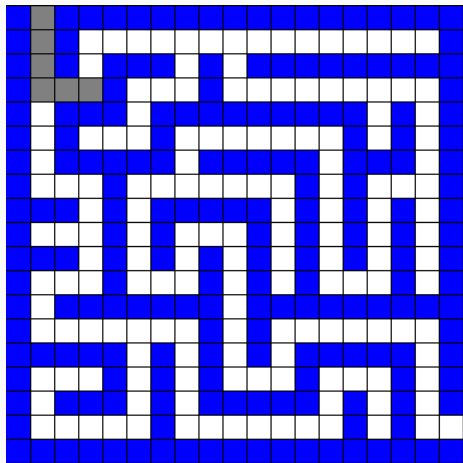
Recorrido de Grafos - DFS



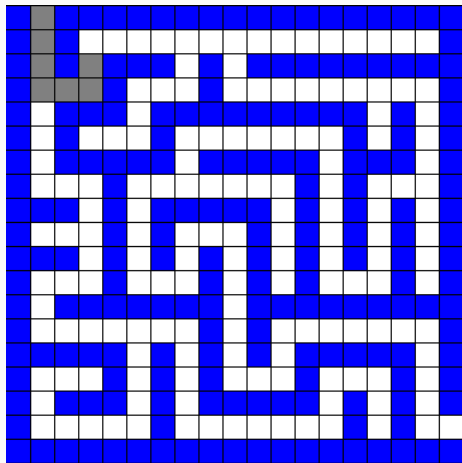
Recorrido de Grafos - DFS



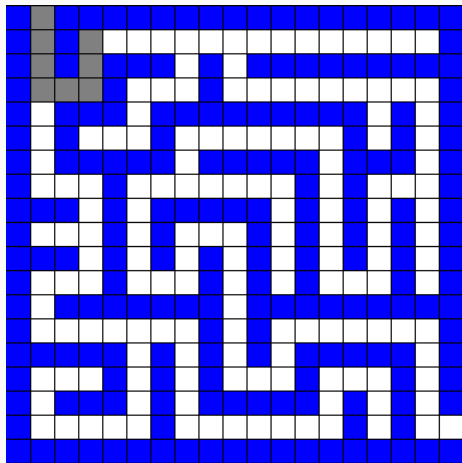
Recorrido de Grafos - DFS

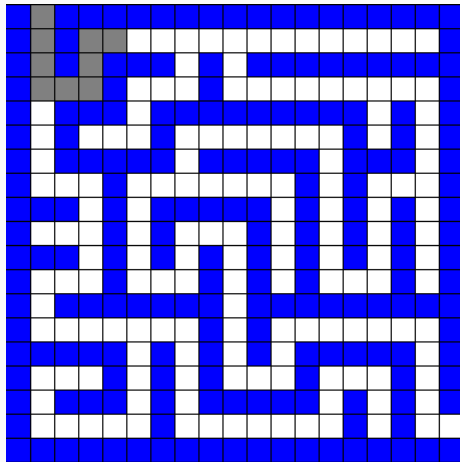


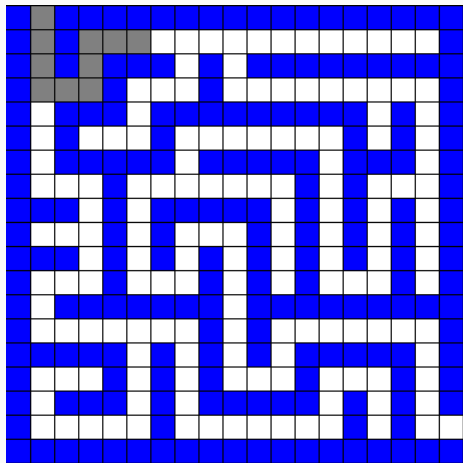
Recorrido de Grafos - DFS

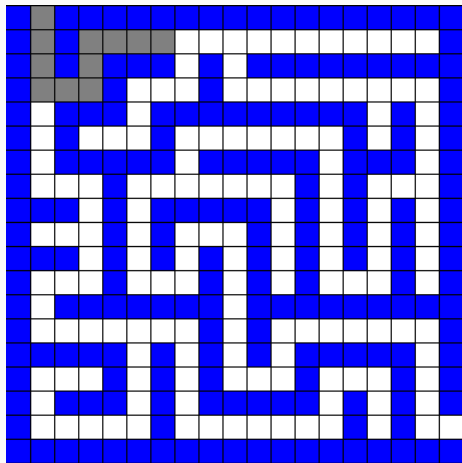


Recorrido de Grafos - DFS

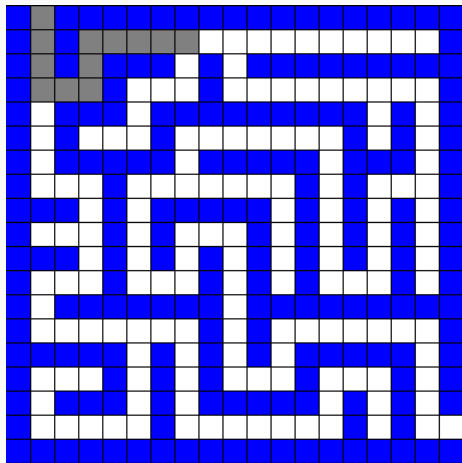




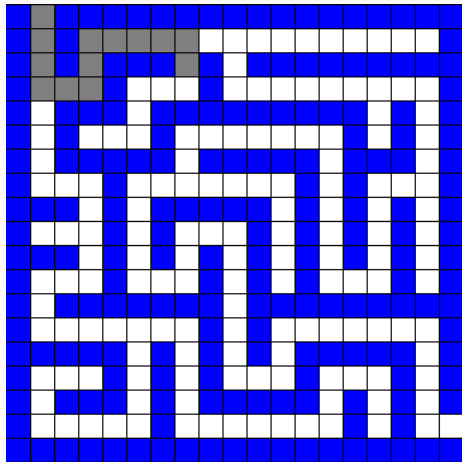




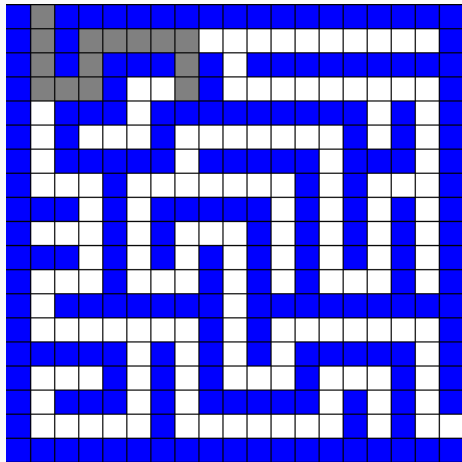
Recorrido de Grafos - DFS



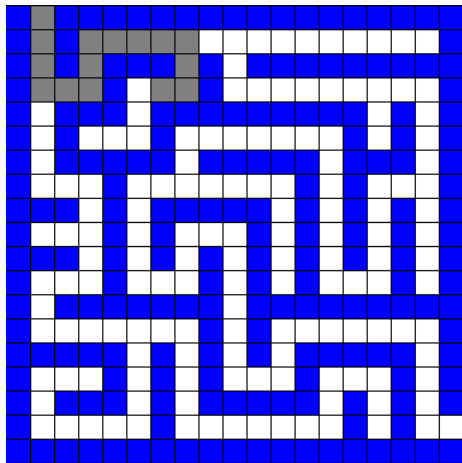
Recorrido de Grafos - DFS



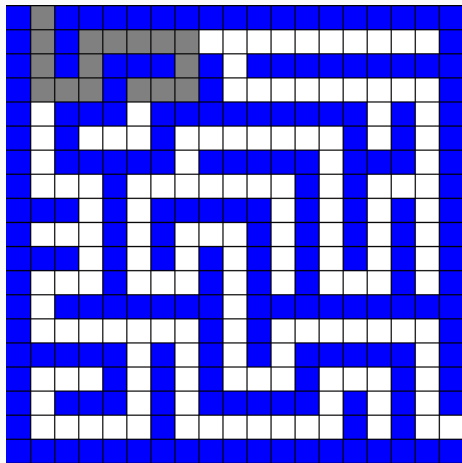
Recorrido de Grafos - DFS



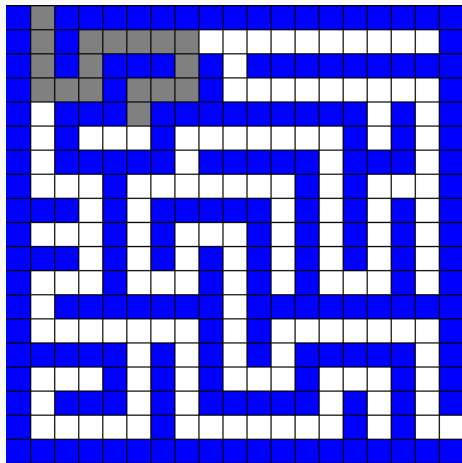
Recorrido de Grafos - DFS



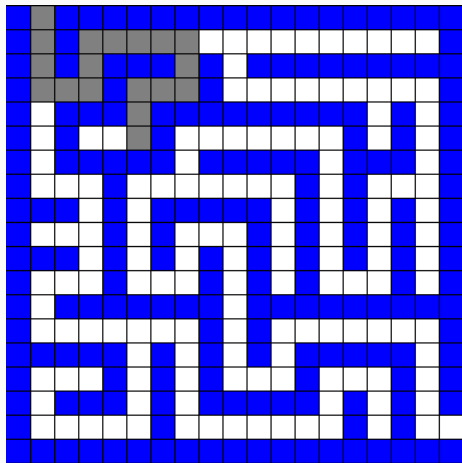
Recorrido de Grafos - DFS



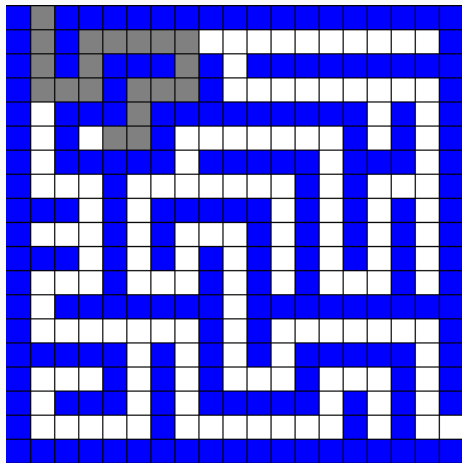
Recorrido de Grafos - DFS



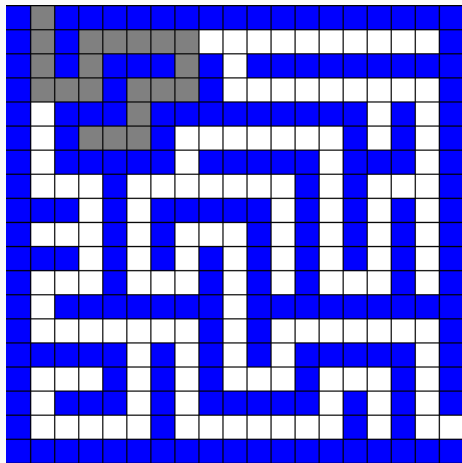
Recorrido de Grafos - DFS



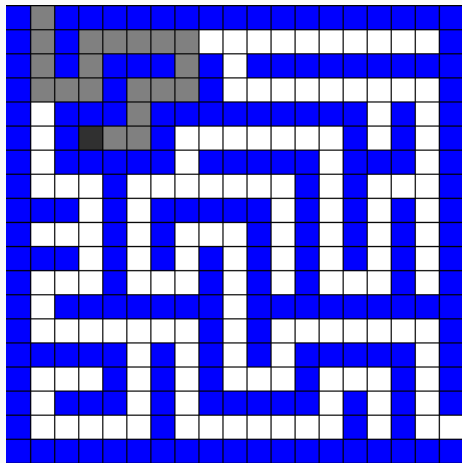
Recorrido de Grafos - DFS

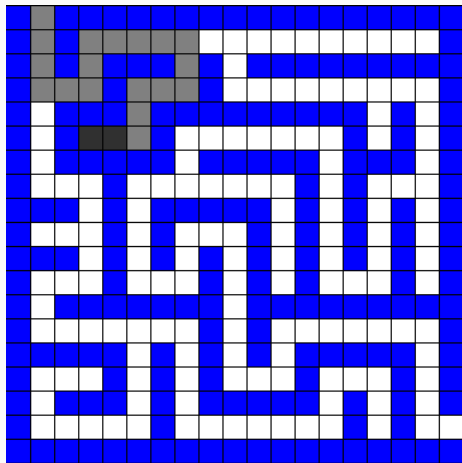


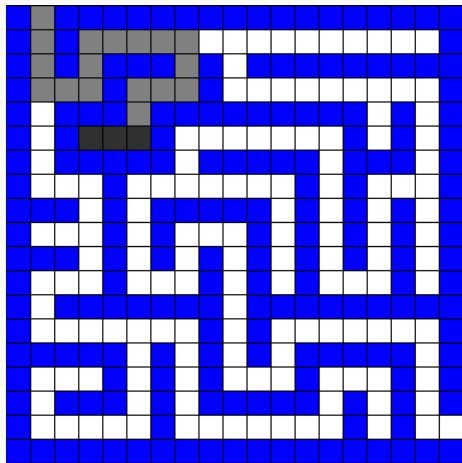
Recorrido de Grafos - DFS

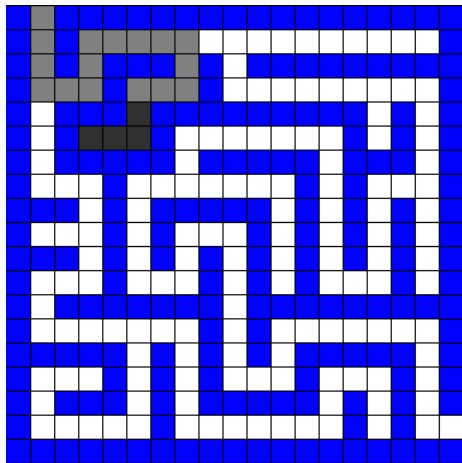


Recorrido de Grafos - DFS

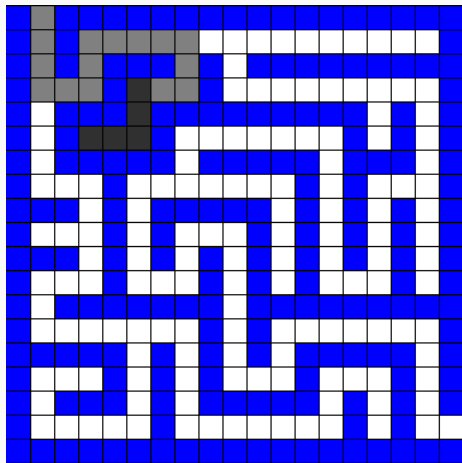


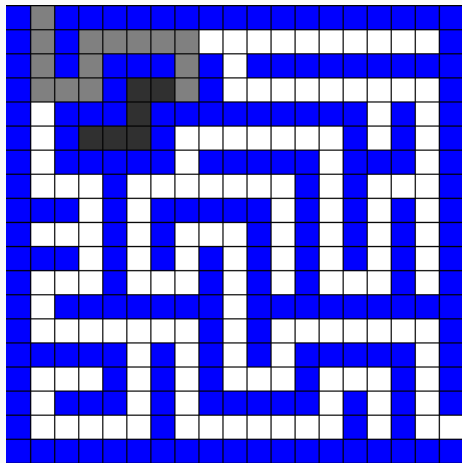




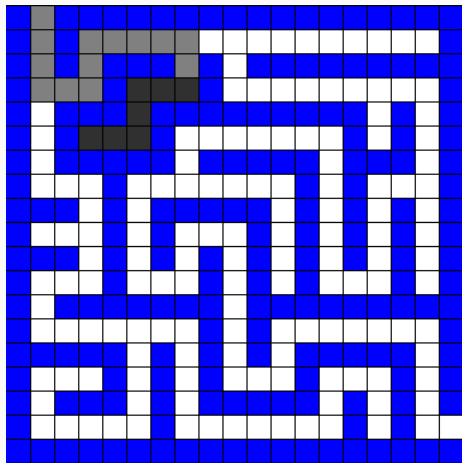


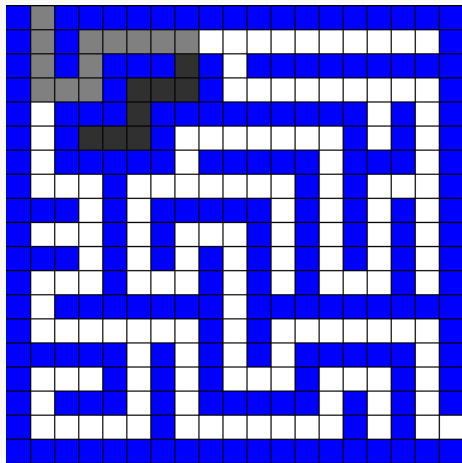
Recorrido de Grafos - DFS

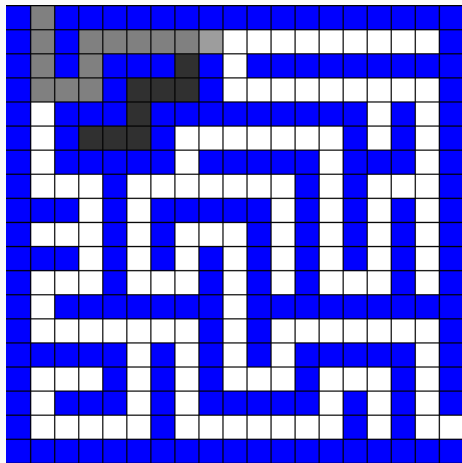




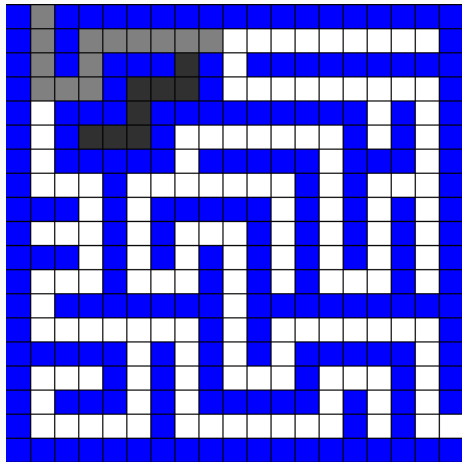
Recorrido de Grafos - DFS



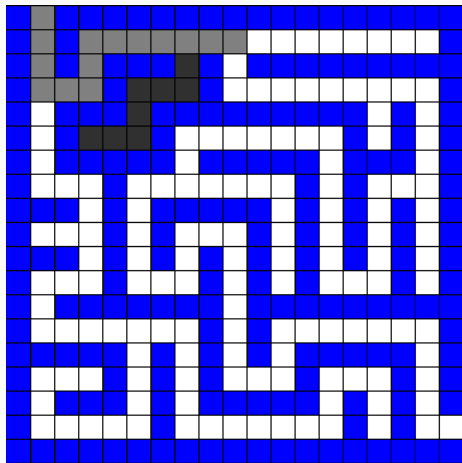


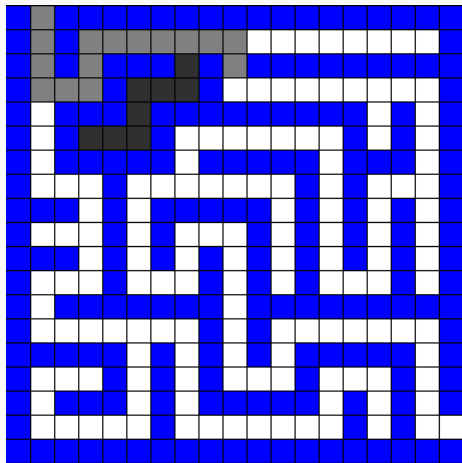


Recorrido de Grafos - DFS

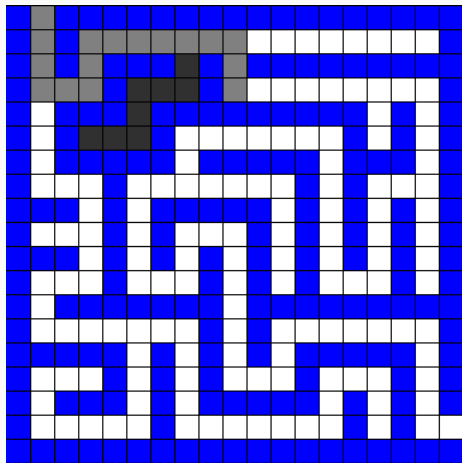


Recorrido de Grafos - DFS

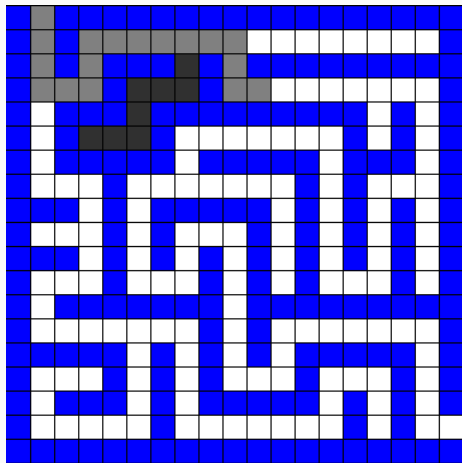




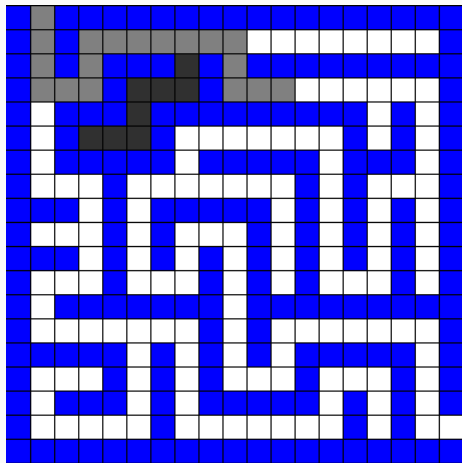
Recorrido de Grafos - DFS

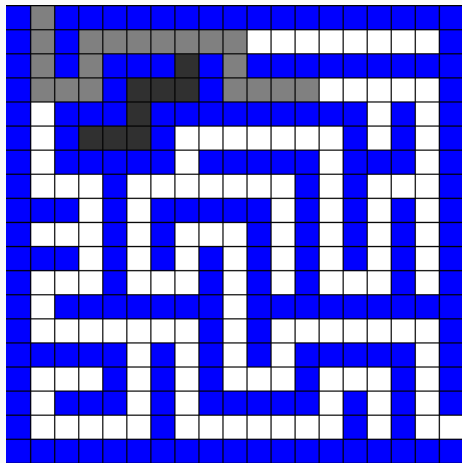


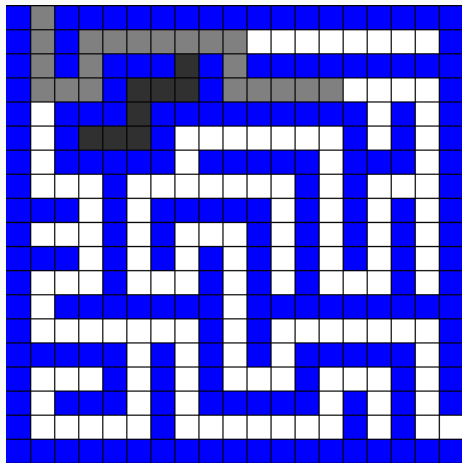
Recorrido de Grafos - DFS



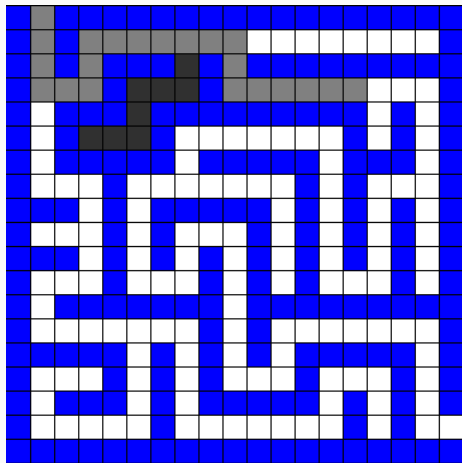
Recorrido de Grafos - DFS

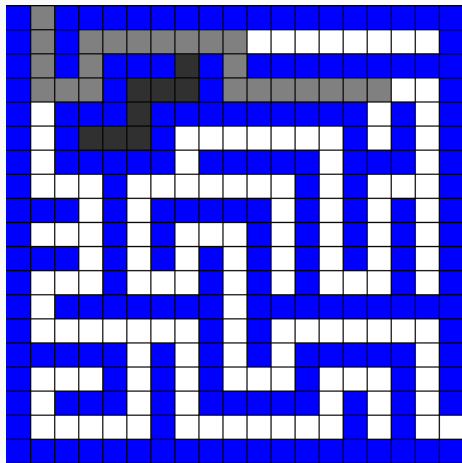


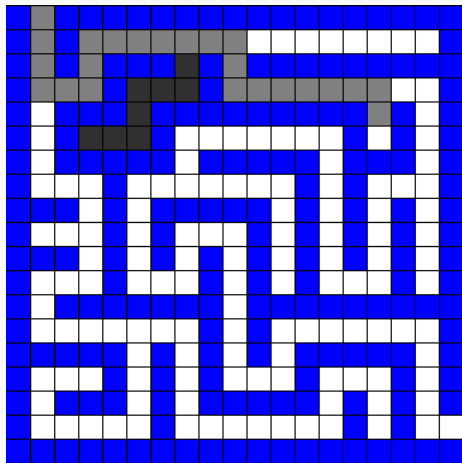




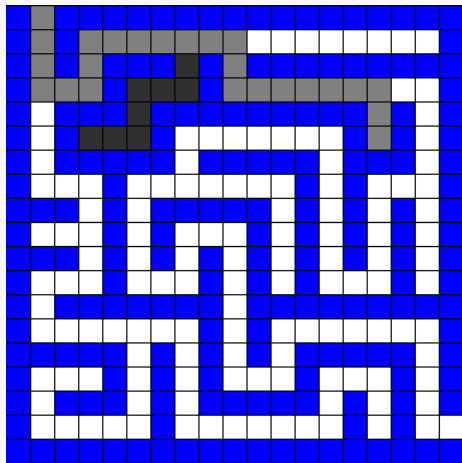
Recorrido de Grafos - DFS

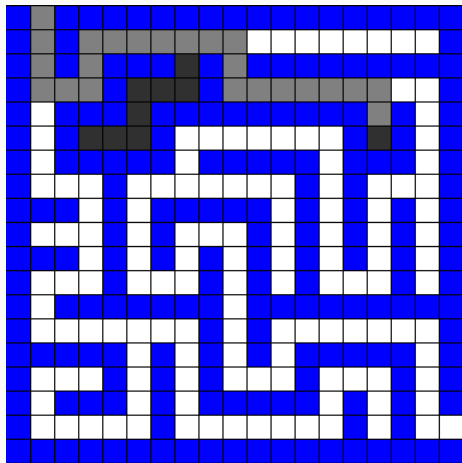


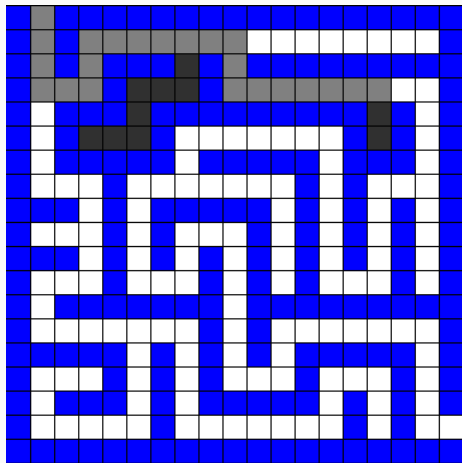


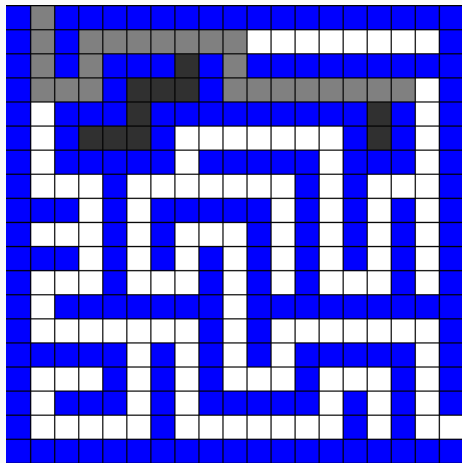


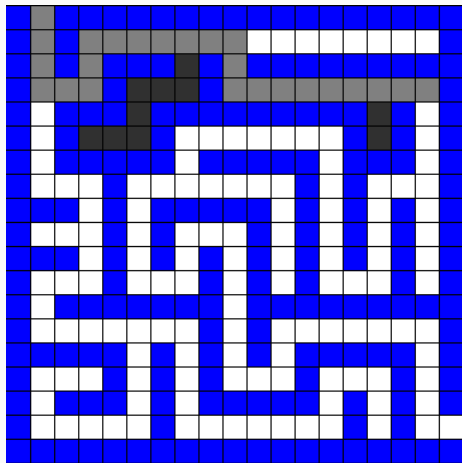
Recorrido de Grafos - DFS



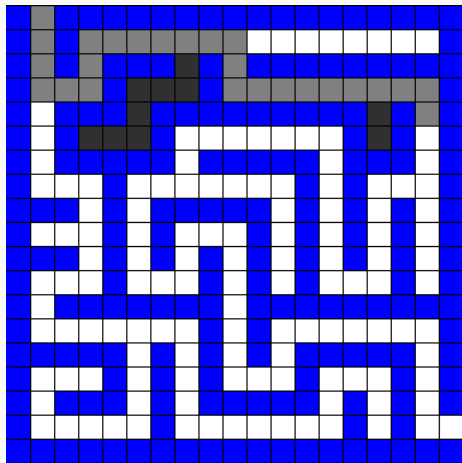


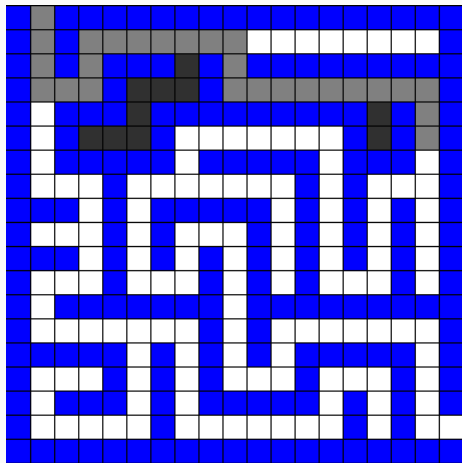






Recorrido de Grafos - DFS





Notemos que DFS-VISITA se llama solo si **visto**[v] = *Falso* e inmediatamente después se tiene que **visto**[v] = *Verdadero* (por cómo funciona el algoritmo), por lo tanto se hacen a lo sumo $\mathcal{O}(|V|)$ llamadas a DFS-VISITA.

Notemos que DFS-VISITA se llama solo si **visto**[v] = *Falso* e inmediatamente después se tiene que **visto**[v] = *Verdadero* (por cómo funciona el algoritmo), por lo tanto se hacen a lo sumo $\mathcal{O}(|V|)$ llamadas a DFS-VISITA.

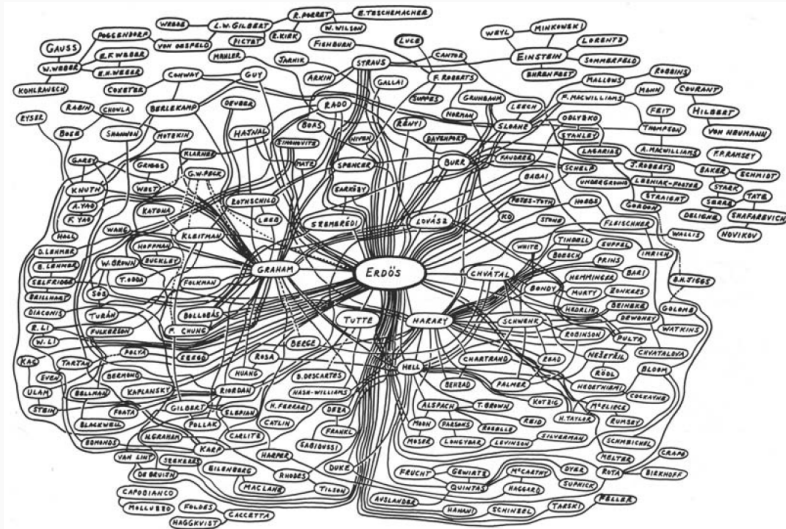
Además, por cada llamada a DFS-VISITA, debemos analizar la situación de todos sus vecinos. Por lo tanto, **la complejidad temporal del algoritmo nos queda $\mathcal{O}(|V| + |E|)$**

Algunos usos del algoritmo DFS para obtener información estructural del grafo incluyen:

- Componentes conexas
- Puentes, puntos de corte, componentes biconexas
- Componentes fuertemente conexas

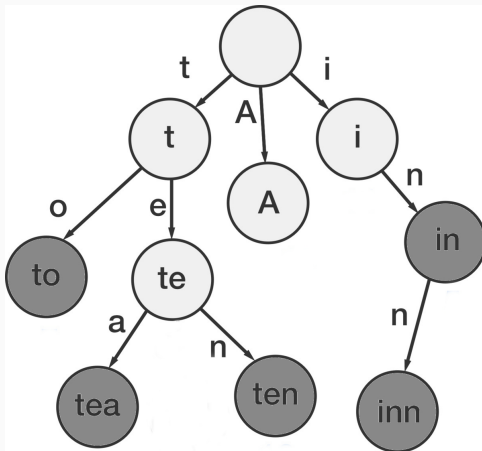
Ejercicio 1

Calcular el número Erdős de Willy, ¿Qué algoritmo usaríamos y cómo lo usaríamos?



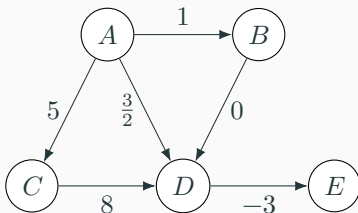
Ejercicio 2

Supongamos que tenemos un Trie en el que están almacenado un conjunto de palabras y se nos pide que demos el listado de palabras que hay en el diccionario en orden lexicográfico. ¿Qué algoritmo usaríamos y cómo lo usaríamos?

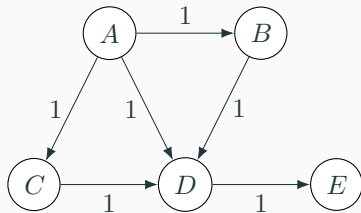


Definición

$G = (V, A, \omega)$ es un digrafo pesado, donde V es el conjunto de vértices, A el conjunto de arcos y ω una función tal que $\omega: A \rightarrow \mathbb{R}$.

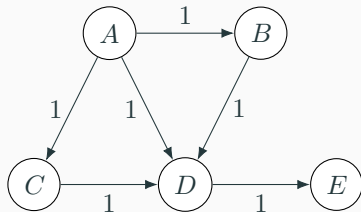


Caminos Mínimos



Para este grafo pesado, ¿qué algoritmo podemos usar para calcular el **camino mínimo** desde *A* al resto de los vértices?

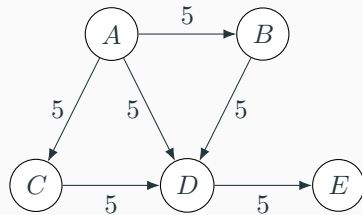
Caminos Mínimos



Para este grafo pesado, ¿qué algoritmo podemos usar para calcular el **camino mínimo** desde *A* al resto de los vértices?

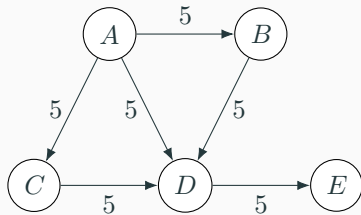
BFS con una pequeña modificación

Caminos Mínimos



¿Y para este?

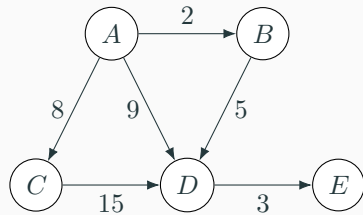
Caminos Mínimos



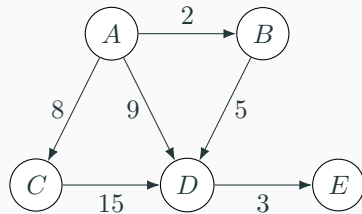
¿Y para este?

BFS con una pequeña modificación

Caminos Mínimos



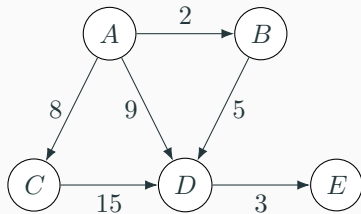
¿Y para este?



¿Y para este?

Dijkstra

Caminos Mínimos

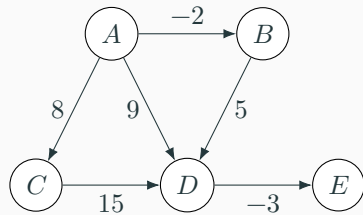


¿Y para este?

Dijkstra

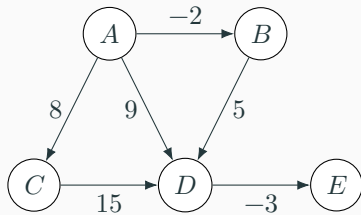
Pero... ¿podría usar BFS?

Caminos Mínimos



¿Y para este?

Caminos Mínimos



¿Y para este?

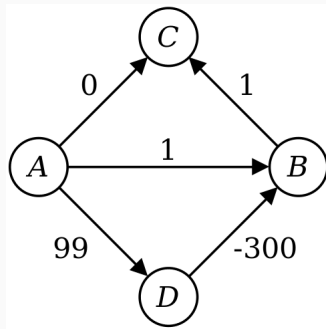
Dijkstra no funciona cuando hay arcos de costo negativo, en este caso necesitamos otro algoritmo (como Bellman-Ford).

Algoritmo de Dijkstra

Dado $G = (V, A, \omega)$ un digrafo pesado, tal que $\omega: A \rightarrow \mathbb{R}_{\geq 0}$, el algoritmo de Dijkstra permite hallar caminos mínimos desde un vértice fuente determinado, con complejidad $\mathcal{O}(|V|^2)$ o $\mathcal{O}(|E| + |V|\log(|V|))$, según cómo se implemente S

Algoritmo de Dijkstra

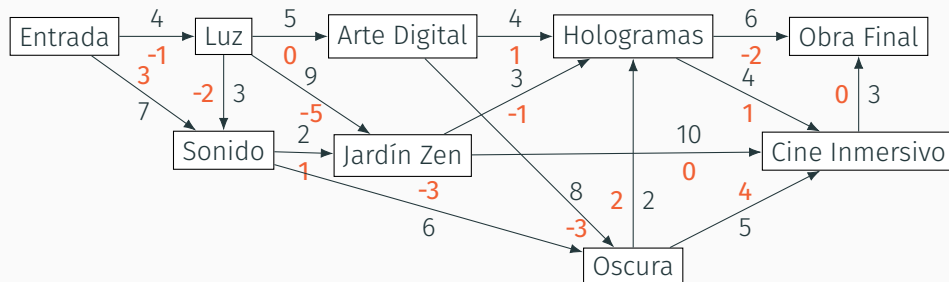
El algoritmo no puede asegurar un resultado correcto cuando hay arcos de costo negativo. Por ejemplo:



Fuente

Algoritmo de Dijkstra - Ejercicio

El Museo de Experiencias Sensoriales quiere desarrollar un sistema de recomendación en tiempo real que sugiera recorridos para visitantes sensibles a distintos tipos de estímulos. cada conexión entre salas tiene un costo sensorial de transición. Además, el museo obtiene datos en tiempo real sobre la congestión de cada sala, que puede aumentar o disminuir el costo percibido por el visitante (en naranja).



1. ¿Cómo buscaríamos un recorrido que minimice el costo?

1. ¿Cómo buscaríamos un recorrido que minimice el costo?
2. ¿Bajo qué supuestos podemos usar Dijkstra?

Algoritmo de Dijkstra - Ejercicio

1. ¿Cómo buscaríamos un recorrido que minimice el costo?
2. ¿Bajo qué supuestos podemos usar Dijkstra?
3. ¿Cómo podríamos determinar un recorrido que visite la mayor cantidad de salas con un límite de estímulo total M ?